

Python Technical Interview Exercise

Project Overview

Please build a RESTful API for a simple task management system (or another domain if you prefer) using Django, Flask, or FastAPI. The API should allow users to:

- Create, read, update, and delete tasks.
- Assign tasks to users. Mark tasks as completed.
- Filter tasks based on status and due date.

Requirements

Backend

- **API Development:**
 - Use **Django (Django REST Framework)**, **Flask**, or **FastAPI**.
 - Implement CRUD endpoints for managing tasks.
 - Use **JWT authentication**.
 - Implement **pagination** for listing tasks.
 - Support filtering tasks by status and due date.
- **Database:**
 - Use **PostgreSQL** or **SQLite** for data storage.
 - Design a simple schema for storing tasks and users.
- **Unit Testing:**
 - Write unit tests for critical endpoints using **pytest/unittest**.
 - Ensure at least **80% test coverage**.
- **Dockerization:**
 - Provide **Dockerfile** and **docker-compose.yml** to setup local app and database.
- **Documentation:**
 - Include a **README.md** with setup instructions and document any key implementation decisions you consider relevant.
 - Provide API documentation using **Swagger (FastAPI)** or **drf-yasg (Django)**.
 - Use of any configuration file (pyproject.toml with poetry or uv), pre-commit or any other good practice that is considered necessary.
- Add **rate limiting** to prevent API abuse.
- Implement **background task processing** (e.g., Celery + Redis).

Frontend

- While the main focus is on Python, we need a decent level of **full stack development** skills, so:

Python Technical Interview Exercise

- Please integrate the backend you just built with a frontend framework of your choice (React, Vue, etc.).
- Key criteria:
 - The frontend should be responsive and user-friendly.
 - Implement CRUD operations associated with the implemented use case.
 - Structured code: Organize your components and state cleanly.
- **Submission Guidelines:**
 - Include a **README** with setup instructions and any other documentation you deem necessary.
 - The application should have seeded data / credentials for demo purposes.

Generative AI tools

Instructions:

- Using your preferred **GenAI coding tool** (e.g., Cursor, Claude Code, Windsurf, GitHub Copilot, etc.), write the **prompt you would use** to generate the API scaffold or full implementation.
- Show the **output code** (or a representative sample of it).
- Describe how you:
 - Validated the AI's suggestions
 - Corrected or improved the output, if necessary
 - Handled edge cases, authentication, or validations
 - Assessed the performance and idiomatic quality of the code

Presentation and Code Review

You will be required to present your project to the technical interview panel. During the presentation, you should explain your user story, design choices, the technical architecture, and demonstrate the functionality of the application. This will be done over Google Meets or Zoom and you will screen share either your GitHub repository or IDE.

After the presentation, the interview panel will conduct a code review of your project. You will be asked to explain your coding decisions and answer any questions related to the code. The interview panel will evaluate your project based on the following criteria:

- **Clean Architecture:** Your architecture should adhere to Clean Architecture principles, including separation of concerns and independence of components.
- **Application testing:** Your project should have sufficient test coverage. Use of TDD is preferable.

Python Technical Interview Exercise

- **Code quality:** Your code should be well-organized, readable, and adhere to best practices.
- **Functionality:** Your application should perform as expected in the requirements without errors or bugs. Optional but desired: no warnings in the browser console.
- **Presentation:** Your presentation should be clear, concise, and demonstrate a good understanding of the project and of the main **backend** and **frontend** best practices.
- **GenAI tools:** Your answers and presentation must show fluency with GenAI tools and prompt engineering, and critical thinking when evaluating AI-generated code.